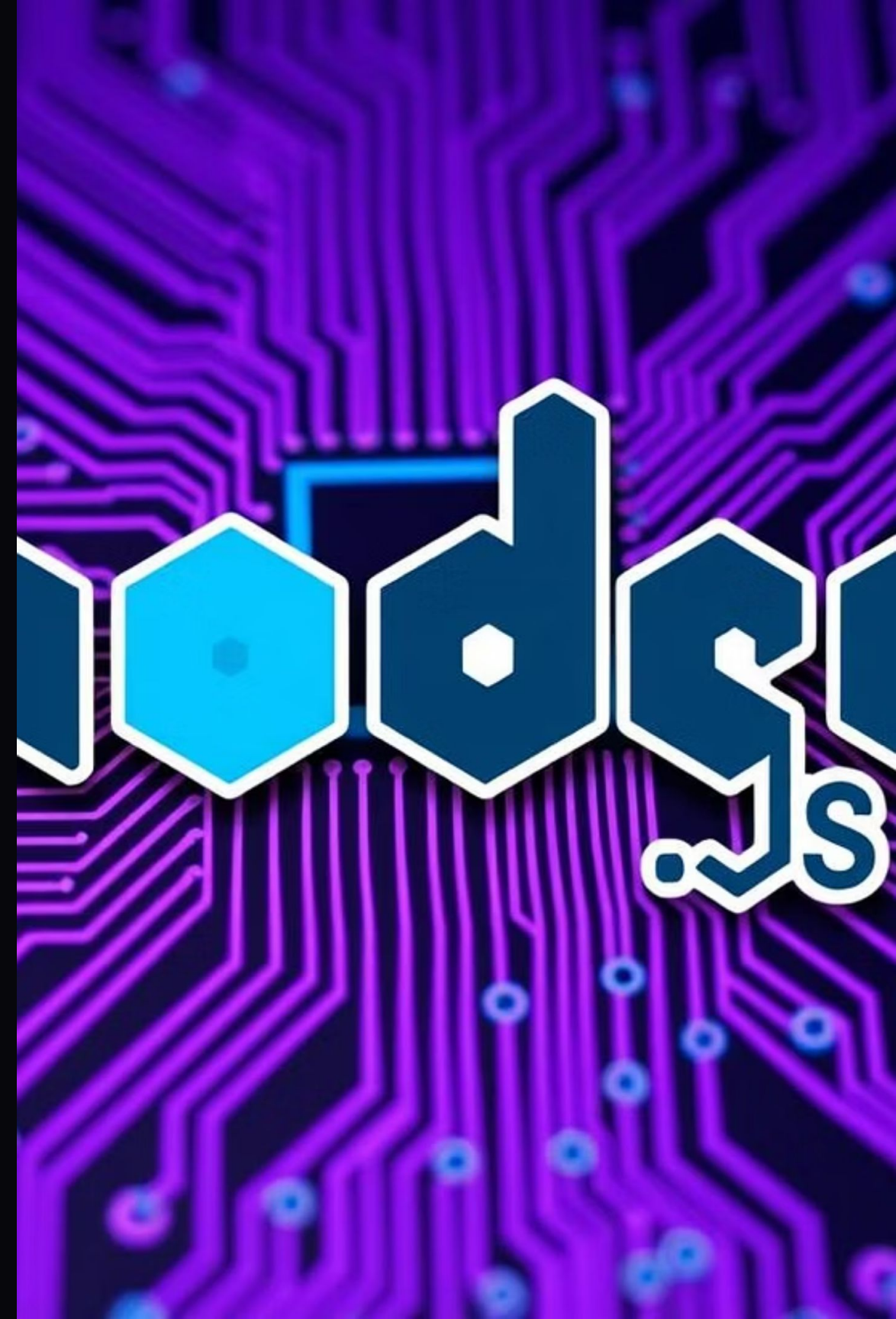


Deep Dive into Node.js: HTTP Module & Beyond

Welcome to Day 2 of our Node.js journey! Today, we're diving deep into the heart of Node.js's networking capabilities: the HTTP module. We'll explore how to build robust web servers, handle various request types, and even serve static files. By the end of this session, you'll have a solid understanding of how Node.js powers the web.



Recap & Introduction to Modules

Day 1: The Foundations

Node.js Fundamentals: A brief overview of what Node.js is – a JavaScript runtime built on Chrome's V8 JavaScript engine, enabling server-side execution.

File System (fs) Module: How to interact with the file system for reading, writing, and manipulating files. Essential for data persistence and loading resources.

Understanding Node.js Modules

Node.js organises functionality into modules, promoting modularity and reusability. This concept is fundamental to building scalable applications.

```
require()
```

Used to import modules into your Node.js files. It allows you to access functionalities exposed by other modules, whether built-in, third-party, or custom-made.



The module system is crucial for structuring Node.js applications, allowing developers to break down complex logic into manageable, reusable components.

HTTP Module In Depth: Building Servers

The `http` module is Node.js's core for handling Hypertext Transfer Protocol. It enables you to create web servers that listen for incoming requests and send responses back to clients.

Creating a Basic HTTP Server

```
const http = require('http');const server =
http.createServer((req, res) => {  res.writeHead(200, {
  'Content-Type': 'text/plain' });  res.end('Hello, Node.js
Server!');});server.listen(3000, () => {  console.log('Server
running on http://localhost:3000');});
```

This foundational setup allows Node.js to act as a web server, capable of responding to browser requests.

Understanding the Components

`http.createServer()`: This function takes a callback that executes for every incoming HTTP request. The callback receives two arguments: `req` (the request object) and `res` (the response object).

`req` (**Request Object**): Represents the incoming request from the client. It contains details like the request URL (`req.url`), HTTP method (`req.method`), headers (`req.headers`), and body content (for POST requests).

`res` (**Response Object**): Used to send data back to the client. Key methods include `res.writeHead()` for setting the HTTP status code and response headers, and `res.end()` for sending the response body and closing the connection.

`server.listen()`: This method starts the HTTP server, making it listen for incoming connections on a specified port (e.g., 3000). Once the server is listening, the provided callback function is executed.

Handling HTTP Methods and Routes

Handling Different HTTP Methods (GET, POST)

```
const http = require('http');const querystring = require('querystring');const server =
http.createServer((req, res) => { if (req.method === 'GET') { res.writeHead(200, {
'Content-Type': 'text/html' }); res.end('<h1>GET Request Received</h1>'); } else if
(req.method === 'POST') { let body = ''; req.on('data', (chunk) => { body += chunk;
}); req.on('end', () => { const parsedData = querystring.parse(body);
res.writeHead(200, { 'Content-Type': 'application/json' });
res.end(JSON.stringify(parsedData)); }); });});server.listen(3000, () => { console.log('Server
running on http://localhost:3000');});
```

The `req.method` property allows you to determine the HTTP method of the incoming request. This is crucial for building RESTful APIs where different actions are performed based on the method (e.g., GET for retrieving data, POST for creating data). For POST requests, data is received in chunks via the `req.on('data')` event and then processed on the `req.on('end')` event, often using modules like `querystring` for parsing URL-encoded bodies or `JSON.parse()` for JSON bodies.

Mastering these concepts is essential for building dynamic web applications with Node.js, allowing your server to respond intelligently to diverse client requests.

Handling Different Routes (URLs)

```
const http = require('http');const server = http.createServer((req, res) => { if (req.url ===
'/') { res.writeHead(200, { 'Content-Type': 'text/html' }); res.end('<h1>Home Page</h1>');
} else if (req.url === '/about') { res.writeHead(200, { 'Content-Type': 'text/html' });
res.end('<h1>About Page</h1>'); } else { res.writeHead(404, { 'Content-Type': 'text/html' });
res.end('<h1>404 Not Found</h1>'); });});server.listen(3000, () => { console.log('Server running
on http://localhost:3000');});
```

The `req.url` property provides the requested URL path. By checking this property, you can implement basic routing, directing requests to different logic based on the URL. This example demonstrates serving different content for the home page (`/`), an about page (`/about`), and a generic 404 "Not Found" response for any other unhandled routes. This manual routing approach is the foundation that frameworks like Express.js build upon.

Working with Query Parameters & Headers

Beyond basic routing, Node.js allows you to extract more granular information from incoming requests, such as query parameters and HTTP headers.

Extracting Query Parameters

```
const http = require('http');const url = require('url');const server =
http.createServer((req, res) => {  const parsedUrl = url.parse(req.url,
true);  const query = parsedUrl.query;  res.writeHead(200, {
'Content-Type': 'application/json' });
res.end(JSON.stringify(query));});server.listen(3000, () => {
console.log('Server running on http://localhost:3000');});
```

Query parameters are key-value pairs appended to the URL after a question mark (e.g., `http://localhost:3000/?name=John&age=30`). The built-in `url` **module** is invaluable here. Using `url.parse(req.url, true)` will parse the URL and return an object, with the `query` property containing an object of the extracted parameters. This allows for dynamic content generation or data filtering based on client input.

Reading Request Headers

```
const http = require('http');const server = http.createServer((req, res)
=> {  const userAgent = req.headers['user-agent'];  res.writeHead(200, {
'Content-Type': 'text/plain' });  res.end(`Your User-Agent:
${userAgent}`);});server.listen(3000, () => {  console.log('Server
running on http://localhost:3000');});
```

HTTP headers provide metadata about the request, such as the client's browser (User-Agent), the content type being sent, or authorisation tokens. The `req.headers` object contains all the incoming headers as key-value pairs. You can access specific headers by their lowercase name, e.g., `req.headers['user-agent']`. This information is crucial for various functionalities, including content negotiation, authentication, and logging.

Understanding and utilising query parameters and request headers allows for more sophisticated server-side logic and interaction with client-side applications.

Making HTTP Requests: Node.js as a Client

Node.js isn't just for building servers; it's also highly capable of acting as an HTTP client, making requests to external APIs or other web services. This is fundamental for integrating your Node.js applications with third-party services.

Performing a GET Request

```
const http = require('http');// GET request to an APIconst options = {  hostname:  'jsonplaceholder.typicode.com',  path:  '/posts/1',  method:  'GET'};const req =  http.request(options, (res) => {  let data = '';  // A chunk of data has been received.  res.on('data', (chunk) => {    data += chunk;  });  // The whole response has been received.  res.on('end', () => {    console.log(JSON.parse(data));  }));});req.on('error', (err) => {  console.error('Error:', err);});req.end();
```

Explanation

1

`http.request(options, callback)`

This method is used to create an HTTP request. The `options` object configures the request with details like the target `hostname`, `path` (the URL endpoint), and `method` (GET, POST, etc.).

2

`res.on('data', ...)`

The response from the external server arrives in chunks. This event listener collects each chunk of data as it's received, appending it to a `data` variable.

3

`res.on('end', ...)`

This event fires once the entire response has been received. At this point, the collected `data` can be fully processed, often by parsing it from JSON string to a JavaScript object using `JSON.parse()`.

4

`req.on('error', ...)`

Essential for robust applications, this listener catches any errors that occur during the request process, such as network issues or invalid hostnames.

5

`req.end()`

Crucially, this method signals that the request has finished being sent. Even for GET requests with no body, `req.end()` must be called to dispatch the request.

Being able to make external HTTP requests allows Node.js applications to consume data from various sources, making them powerful intermediaries in complex service architectures.

Key Takeaways & Next Steps

Today, we've significantly expanded our understanding of Node.js's core capabilities, particularly in handling HTTP communications.

Key Takeaways

HTTP Server Mastery: We built an HTTP server from scratch, understanding the roles of `req` and `res` objects.

Dynamic Request Handling: Learned to differentiate and respond to various HTTP methods (GET, POST) and implement basic routing based on `req.url`.

Data Extraction: Explored how to extract dynamic data from URLs using query parameters with the `url.parse` module and read important metadata from request headers like `User-Agent`.

Static File Serving: Developed a simple static file server, essential for serving front-end assets (HTML, CSS, JS, images) and implementing custom 404 pages.

HTTP Client Capabilities: Demonstrated how Node.js can make external HTTP requests to interact with other APIs, showcasing its versatility.

Looking Ahead: Day 3

While the built-in `http` module is powerful, directly managing complex routing and middleware can become cumbersome. Tomorrow, we'll simplify our development process.



Express.js

Discover Express.js, a minimalist web framework for Node.js that streamlines server development, routing, and middleware integration.



Middleware & REST APIs

Understand the concept of middleware for request processing and delve into building structured RESTful APIs with Express.



Connecting to Databases

Learn how to integrate your Node.js applications with databases, focusing on MongoDB for persistent data storage.

Prepare for a session that will transition your Node.js server-side skills from foundational to framework-driven, unlocking the potential for scalable and maintainable web applications.